At this point, after the API call, the response should hold the information for the required function calls. Remember, we asked the LLM to provide us with three recommendations, which means it should also provide us with three function call outputs, as shown in the following listing.

Listing 5.6 `parallel_functions.py` (`run_conversation`, `tool_calls`)

```
tool_calls = response_message.tool_calls     #1
if tool_calls:    #1
    available_functions = {
        "recommend": recommend,
    }     #2
    # Step 4: send the info for each function call and function response to
the model
    for tool_call in tool_calls:     #3
        function_name = tool_call.function.name
        function_to_call = available_functions[function_name]
        function_args = json.loads(tool_call.function.arguments)
        function_response = function_to_call(
            topic=function_args.get("topic"),     #4
            rating=function_args.get("rating"),
        )
        messages.append(     #5
            {
                "tool_call_id": tool_call.id,
                "role": "tool",
                "name": function_name,
                "content": function_response,
            }
        )  # extend conversation with function response
    second_response = client.chat.completions.create(     #6
        model="gpt-3.5-turbo-1106",
        messages=messages,
    )
    return second_response.choices[0].message.content  #6
```

#1 If the response contains tool calls, execute them.
#2 Only one function but could contain several
#3 Loops through the calls and replays the content back to the LLM
#4 Executes the recommend function from extracted parameters
#5 Appends the results of each function call to the set of messages
#6 Sends another request to the LLM with updated information and returns the message reply

The tool call outputs and the calls to the recommender function results are appended to the messages. Notice how messages now also contain the history of the first call. This is then passed back to the LLM to construct a reply in natural language.

Debug this example in VS Code by pressing the F5 key with the file open. The following listing shows the output of running `parallel_functions.py`.